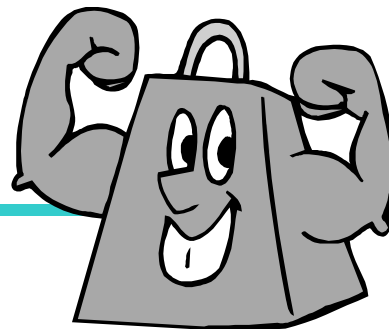




算法与算法方法-

穷举算法





蛮力算法

- 蛮力算法用最直接的方式求解问题, 一点不考虑如何使算法更有效.
- 蛮力算法的例子有穷举算法、线性搜索, 冒泡排序和插入排序等。

穷举算法思路:暴力搜索

- 穷举法解题

解题思路：对可能是解的众多 候选解 按某种顺序进行 逐一列举 和逻辑检验，从中找出那些符合要求的候选解作为问题的解。

- 如：密码箱忘记密码怎么办？
- 穷举算法的设计核心：
 - 定义所有候选解的集合，即“解空间”；

解空间1：生成排列问题

例1：编写程序，求出所有5、6、7组成的、且各位数字互不相同的三位数。

(1)定义所有的排列集合：

$$\{a_1a_2a_3 \mid a_1, a_2, a_3 \in \{5, 6, 7\}\}$$

百位可以是5,6,7

十位可以是5,6,7

个位可以是5,6,7

(2)检查是否满足约束条件

百、十、个位互不相同

排列问题求解算法:

Begin

定义位置变量high,mid,low;

穷举百位的所有方案 high:=5 to 7

穷举十位的所有方案 mid:=5 to 7

穷举个位的所有方案 low:=5 to 7

if (high、mid和low各不相同)

打印(high*100+mid*10+low);

End

```
#include <stdio.h>
main(){
    int high,mid,low;    int count=0;
    for(high=5;high<=7;high++)
        for(mid=5;mid<=7;mid++)
            if(high!=mid)
                for(low=5;low<=7;low++)
                    if(low!=high && low!=mid){
                        count++;
                        printf("%d\t",high*100+mid*10+low);
                        if(count%3==0) putchar('\n');
                    }
    return 0;
}
```

总结生成排列问题算法

- 生成排列问题的通用算法：按排列位置和可选字符进行穷举。
- 如果题目要求列出不连续的几个符号a、b、c、d...的所有排列情况呢？例：sports可以组成多少变位词？
 - 算法主体不变，用整数1..n代表n个符号，然后穷举。找到一个排列后,用switch结构将数字对应为符号。

但是循环嵌套的列举个数是有限的，因此更高效的生成排列算法需要找出排列的数学规律。

解空间2：百鸡问题

求解百鸡问题：用100元买100只鸡，大公鸡5元1只，母鸡3元1只，小鸡1元3只。问各能买多少只？

(1)解空间是三元一次方程的所有非负整数解；

$$\{(x, y, z) \mid 5x + 3y + z/3 = 100 \text{ and } x, y, z \in N\}$$

公鸡只数 x 的范围是0..20

母鸡只数 y 的范围是0..33

小鸡只数 z 的范围是 $0 \leq z < 100$

(2)检查是否满足约束条件

$$5x + 3y + z/3 = 100 \text{ and } x + y + z = 100$$

百鸡问题的算法分析:

begin

定义整数变量cocks,hens,chicken;

穷举公鸡的所有方案 cocks:=0 to 20

穷举母鸡的所有方案 hens:=0 to 33

穷举小鸡的所有方案 chicken:=0 to 100

if (只数为100且钱数为100)

打印cocks,hens,chicken;

end

```
#include<stdio.h>
int main()
{
    int cocks,hens,chicken;
    for(cocks=0;cocks<=20;cocks++)
        for(hens=0;hens<=33;hens++)
            for(chicken=0;chicken<=100;chicken+=3)
                if (cocks+hens+chicken==100 &&
                    cocks*5+hens*3+chicken/3==100)
                    printf("cocks:%d,hens:%d,chicken:%d\n",cocks,h
                        ens,chicken);
    return 0;
}
```

总结 n 元一次方程求解算法

- 类似百鸡问题，题目要求的候选解众多，且解空间是 n 元一次方程的非负整数解。
- 算法：逐一穷举 n 元的取值范围内所有可能值，然后检验每个候选解是否满足方程。

解空间3:寻找特殊数

- 例2: 打印所有除以11后所得商正好是它的各个数字平方和的三位数。

算法分析:

定义整数变量n和百位、十位、个位变量d1,d2,d3;

穷举三位数n的所有方案 $n:=100$ to 999

分解n的百位 $d1=n/100$;

分解n的十位 $d2=(n/10)\%10$;

分解n的个位 $d3=n\%10$;

if ($n/11 ==$ 数码平方和)

打印当前的n;

3位数中除11后商等于各数字平方和

```
#include<stdio.h>
int main(){
    int i,d1,d2,d3,count;

    for (i=100,count=0; i<=999; i++){
        d1=i/100;
        d2=(i/10)%10;
        d3=i%10;
        if (i/11 == (d1*d1 + d2*d2 + d3*d3)){
            count++;    printf("%d\t", i);
            if(count%3==0)    putchar('\n');}
    }
    return 0;
}
```

总结：寻找数位分解的特殊数

例3：打印1..n之间所有除以m后所得商正好是它的各个数字平方和的数。

算法分析：

定义整数变量n,m,i,squareSum,tmp,digit;

接受键盘输入的n和m;

for i:=1 to n

for(tmp=i,squareSum=0; tmp!=0; tmp/=10)

digit = tmp%10 ;

squareSum +=digit*digit;

{从低位分解整数i，计算各位数平方和}

if (i/m == squareSum)

print i;



1..n之间除m商等于各数字平方和

```
#include<stdio.h>
int main()
{
    int n,m,i,squareSum,tmp,digit,count;
    scanf("%d%d",&n,&m);
    for (i=1,count=0; i<=n; i++)
    {
        for(tmp=i,squareSum=0; tmp!=0; tmp/=10)
        {
            digit = tmp%10;
            squareSum += digit*digit;
        }
        if (i/m == squareSum){
            count++;    printf("%d\t", i);
            if(count%3==0)    putchar('\n');}
    }
    return 0;
}
```

练习：一个大于等于3的正整数，判断它是否为一个素数（素数）。

◆所谓素数，是指除了1和该数本身外，不能被其他任何整数整除的数。例如，13是素数（素数）。因为它不能被2，3，4，...，12整除。

◆由于素数在当代的密码学中扮演了中心的作用，所以该问题具有重要意义。

◆判断一个数 n ($n \geq 3$) 是否为素数：将 n 作为被除数，将 $2..n-1$ 各个整数依次作为除数，如果都不能被整除，则 n 为素数。


```
procedure isPrime(n:  $\geq 3$  integer)
  for i:=2 to n-1
    if (n%i==0)
      break;
    i++;
  if (i<n)
    print n不是素数
  else
    print n是素数
  return
```

提示：n为素数的穷举因子范围，只需2.. 之间即可。

```
#include<stdio.h>
int main(){
    int n,i,isPrime;
    scanf("%d",&n);
    for(i=2,isPrime=1;i*i<=n && (isPrime==1); i++){
        if (n%i==0)
            isPrime=0;
    }
    if (isPrime)
        printf("Yes\n");
    else
        printf("No\n");
    return 0;
}
/* isPrime==1 可以简写为 isPrime*/
```

解空间5：逻辑谜题

例4：5位跳水高手将参加10m高台跳水决赛，好事者让5人据实力预测比赛结果。

A选手说：B第二，我第三；

B选手说：我第二，E第四；

C选手说：我第一，D第二；

D选手说：C最后，我第三；

E选手说：我第四，A第一。

决赛成绩公布后，每位选手的预测都说对了一半，即一对一错。请设计算法求出比赛的实际名次。

逻辑谜题的解空间：n元组的逻辑判断

- 定义逻辑谜题的候选解空间：{ABCDE的所有排列}
 - 但是重要的不是打印排列，而是名次；
 - 解空间改为{ABCDE的名次排列12345}；
 - 穷举A的名次排列 a 1..5,
 - 穷举B的名次排列 b 1..5,
 - 穷举C的名次排列 c 1..5,
 - 穷举D的名次排列 d 1..5,
 - 穷举E的名次排列 e 1..5,

太长的逻辑判断化简方法：整数运算

- 检测条件:

A说: B第二, A第三;

B说: B第二, E第四;

C说: C第一, D第二;

D说: C第五, D第三;

E说: E第四, A第一。

而且都只说对了一半。

```
(b==2 && a!=3 || b!=2 && a==3) &&
```

```
(b==2 && e!=4 || b!=2 && e==4) &&
```

```
(c==1 && d!=2 || c!=1 && d==2) &&
```

```
(c==5 && d!=3 || c!=5 && d==3) &&
```

```
(e==4 && a!=1 || e!=4 && a==1)
```

```
countA=(b==2)+(a==3);
```

```
countB=(b==2)+(e==4);
```

```
countC=(c==1)+(d==2);
```

```
countD=(c==5)+(d==3);
```

```
countE=(e==4)+(a==1);
```

```
if (countA==1 && countB==1 && countC==1  
    && countD==1 && countE==1)
```

比赛名次程序(1)

```
#include <stdio.h>
int main()
{   int a,b,c,d,e;//用于记录A~E分别的名次
    int countA,countB,countC,countD,countE;//用于记录
    A~E分别说对的话个数

    for(a=1;a<=5;a++)//a~e分别代表A~E选手的名次
        for(b=1;b<=5;b++)
            if (a!=b)
                for(c=1;c<=5;c++)
                    if (c!=a &&c!=b)
                        for(d=1;d<=5;d++)
                            if (d!=a && d!=b && d!=c)
                                for(e=1;e<=5;e++)
```

```
if (e!=a && e!=b && e!=c && e!=d){
    countA=(b==2)+(a==3);
    countB=(b==2)+(e==4);
    countC=(c==1)+(d==2);
        countD=(c==5)+(d==3);
        countE=(e==4)+(a==1);
    if (countA==1 && countB==1 && countC==1 &&
countD==1 && countE==1){
        printf("比赛名次是:\n");
        printf("A:第%d名\nB:第%d名\nC:第%d名\nD:第%d
名\nE:第%d名\n",a,b,c,d,e);
    } //if
}
}
```

挑战：整数素因子分解

- 在许多情况下，知道一个数是否是素数还不够，有时，还需要知道它的素因子序列。每个大于1的正整数都能表示为素数的乘积。这个因数分解是唯一的，被称为素数分解式（prime factorization）。
- 例如，数字 $60=2*2*3*5$, $799=17*47$, 其中每一个因子都是素数。
- 注意，同一个素数在因数分解中可以出现多次。

- 问题分析: $60=2*2*3*5$
- $f(n)=p*f(n/p)$, p 是 n 的最小素数因子
- 对 n 进行素数分解, 可以转化为两步:
 - 1: 找到 n 的最小素数因子 p ; (穷举)
 - 任意整数 n 的最小因子一定是素数; 因此, 找到最小因子即可, 不必再做素数判定。
 - 2: 对 n/p 继续进行素数分解; (迭代)

Procedure primefactor(n : >3 integer)

$q = n$;

while $q \neq 1$ {迭代条件}

 for $i := 2$ to q {穷举找出最小因子}

 if ($q \% i == 0$)

 打印素因子 i 和乘号 '*';

 break;

$q = q/i$; {迭代公式}

整数的素因子分解v1

```
#include <stdio.h>
int main(){
    int n,q, i,find;
    scanf("%d",&n);
    for (q=n; q!=1; ){
        for (i=2; i<=q && (q%i!=0);i++) ;
        if (q%i==0){
            printf("%d",i);
            if (q!=i) putchar('*');
        }
        q = q/i;
    }
    return 0;
}
```

Procedure primefactor(n : >3 integer)

$q := n$

while $q \neq 1$ {迭代条件}

~~for~~ $i := i$ to q {素因子分解式保持非递减次序}

if ($q \% i == 0$) {穷举素因子的检测条件}

print $i + '*'$;

$q = q / i$; {迭代公式}

else

$i++$; {内层循环由此句代替}

整数的素因子分解v2

```
#include<stdio.h>
int main(){
    int n, q, i;
    scanf("%d",&n);
    for(q=n,i=2; q!=1;){
        if (q%i==0){
            printf("%d",i);
            if (q!=i) putchar('*');
            q=q/i;
        }
        else i++;
    }
    return 0;
}
```

例 3：A、B、C、D、E 五人夜间合伙捕鱼，凌晨时都疲倦不堪，各自在河边的树丛中找地方睡着了。日上三竿，A 第一个醒来，他将鱼分作五份，把多余的一条扔回河中，拿自己的一份回家去了。B 第二个醒来，也将鱼分作五份，扔掉多余的一条，拿走自己的一份，接着 C、D、E 依次醒来，也都按同样的办法分鱼，问五人至少合伙捕了多少条鱼？试编程序算出。

算法分析：迭代+穷举

问题分析：

设原先鱼的总数是 x 条，设A、B、C、D、E拿之前剩下的鱼数量分别为A、B、C、D、E，则

$$B = (A-1) * 4/5; \dots E = (D-1) * 4/5;$$

$$\text{反推得 } D = E * 5/4 + 1; C = D * 5/4 + 1; B = C * 5/4 + 1; A = B * 5/4 + 1;$$

以上是总结出的迭代公式。

若E已知，则由E可依次推出D、C、B、A。E最小是6，但是6不能保证D、C、B、A计算过程的整除特性。

因此，需要对E进行穷举，E可以是6、11、16……（以5递增），穷举是为了找到满足题目约束的最小E值：

$$(D-1) \bmod 5 == 0$$

$$(C-1) \bmod 5 == 0$$

$$(B-1) \bmod 5 == 0$$

$$(A-1) \bmod 5 == 0$$

捕鱼问题求解算法：

begin

定义五个人所分鱼的数量 a, b, c, d, e ;

穷举 e 的所有方案 $e := 6$ to 直到有一个最小解

迭代计算 $d = e * 5 / 4 + 1$; 检测 $(e - 1) \% 5 == 0$;

迭代计算 $c = d * 5 / 4 + 1$; 检测 $(d - 1) \% 5 == 0$;

迭代计算 $b = c * 5 / 4 + 1$; 检测 $(c - 1) \% 5 == 0$;

迭代计算 $a = b * 5 / 4 + 1$; 检测 $(b - 1) \% 5 == 0$;

若以上规则都满足，打印 a 值;

否则 $e := e + 5$;

end

捕鱼问题程序代码

```
#include<stdio.h>
int main(){
    int e; //e所分的鱼数量
    int n; //迭代a~d的鱼数量
    int isOK; //鱼数符合规则的次数
    int i;

    for (e=6,isOK=0; isOK<4; e+=5) { //穷举e
        for (i=1,n=e,isOK=0; i<=4; i++){ //迭代求d~a
            n = n*5/4+1;
            if ((n-1)%5==0)
                isOK++;
        }
    }
    printf("fish=%d\n",n);
    return 0;
}
```

- 问题描述：两个乒乓球队进行对抗赛，甲队出A、B、C、D四人；乙队出W、X、Y、Z四人。已知部分抽签结果是：A不与X比赛；C不与X、Z比赛，D不与W比赛。你的任务是将全部可能的抽签结果打印出来。

Group 1:

A VS W

B VS X

C VS Y

D VS Z ...

抽签结果：按选手组合

- 解空间：穷举甲乙两队对抗的所有组合，逐一检验；去掉(A,X),(C,X),(C,Z),(D,W)组合.
- 解空间设计方式一：

{从两队中任选两人对抗的所有情况}.

```
for (i='A'; i<='D'; i++)  
    for (j='W'; j<='Z'; j++)  
        { ... }
```

但这样的穷举不能满足输出要求将两队四局对抗组合打印。

抽签结果：按甲队排列

- 解空间：穷举甲乙两队对抗的所有组合，逐一检验去掉(A,X),(C,X),(C,Z),(D,W)组合.
 - 解空间设计方式二：
 - {甲队ABCD四人对抗选手的所有排列}
- ```
for (a='W'; a<='Z'; a++)
 for (b='W'; b<='Z'; b++)
 for (c='W'; c<='Z'; c++)
 for (d='W'; d<='Z'; d++)
```

```
#include<stdio.h>
int main(){
 int a,b,c,d,group=0;
 for (a='W'; a<='Z'; a++)
 { if (a!='X')
 for (b='W'; b<='Z'; b++)
 { if (a!=b)
 for (c='W'; c<='Z'; c++)
 {if (c!=a && c!=b && c!='X' && c!='Z')
 for (d='W'; d<='Z'; d++)
 {if (d!=a && d!=b && d!=c &&
d!='W')
```

```
{ group++;
 printf("Group %d:\n",group);
 printf("A VS %c\n",a);
 printf("B VS %c\n",b);
 printf("C VS %c\n",c);
 printf("D VS %c\n",d);
 }//end if (d)
 }//end for(d)
} //end for(c)
} //end for(b)
} //end for(a)
printf("%d\n",group);
return 0;
}
```

## 练习：方格棋盘

问题描述：已知正整数 $n$ 、 $m$ ，设有一个 $n*m$ 方格的棋盘（ $1 \leq n \leq 100$ ， $1 \leq m \leq 100$ ）。求出该棋盘中包含正方形与长方形（长方形中不包括正方形）的个数。

例如：当 $n=2$ ， $m=3$ 时，

正方形的个数有8个，即边长为1的正方形有6个；边长为2的正方形有2个。

长方形的个数有10个，即 $2*1$ 的长方形有3个； $1*2$ 的长方形有4个； $1*3$ 的长方形有2个； $2*3$ 的长方形有1个。

分析：

边长=1的正方形 $n*m$ 个，边长=2的正方形 $(n-1)*(m-1)$ 个，...

边长= $\min(n,m)$ 的正方形 $(n-\min+1)*(m-\min+1)$ 个。

2\*1的长方形 $(n-1)*m$ 个，

1\*2的长方形 $n*(m-1)$ 个，...

穷举所有边长：正方形1.. $\min$ ，长方形 (1.. $n$ , 1.. $m$ )。

再累加： $f = f + (n - \text{edgeA} + 1) * (m - \text{edgeB} + 1)$ 。



## 练习：方格棋盘

```
int main()
{
 int n,m,edgeA,edgeB,min,sumSq,sumRec;
 scanf("%d%d",&n,&m);
 min = (n>m)? m : n;//if (n>m) min=m; else min=n;
 for (edgeA=1,sumSq=0; edgeA<=min; edgeA++)
 sumSq += (n-edgeA+1)*(m-edgeA+1);
 printf("squares have %d,",sumSq);
 for (edgeA=1,sumRec=0; edgeA<=n; edgeA++)
 for (edgeB=1; edgeB<=m; edgeB++)
 {
 if (edgeA!=edgeB)
 sumRec += (n-edgeA+1)*(m-edgeB+1);
 }
 printf("Rectangles have %d\n",sumRec);
 return 0;
}
```

## ? : 条件运算符

上例中运算表达式  $\text{min} = (n > m) ? m : n;$

含义是 if (n > m)

    min = m;

else

    min = n;

条件运算符?:的C语法是

(条件表达式) ? 表达式1 : 表达式2

运算规则是：当条件表达式为真，运算结果为表达式1，否则为表达式2.

